

Wprowadzenie do sprzętowych sieci neuronowych

Prowadzący: dr inż. Andrzej Skoczeń

D-10 222, tel. w. 28-72,

e-mail: skoczen@agh.edu.pl

Laboratorium 2

2026

22 kwiecień 2026 grupa 1

8 maj 2026 grupa 2

Przebieg projektu

1. Zaloguj się do systemu UBUNTU Linux na module Kria
2. Utwórz projekt blokowy w Vivado
3. Dodaj klasyfikator oparty na kodzie Verilog z sieci neuronowej
4. Pobierz trzy pliki: .xsa, .bit oraz skrypt TCL projektu
5. Uruchom interfejs Jupyter i sprawdź, jak działa klasyfikator

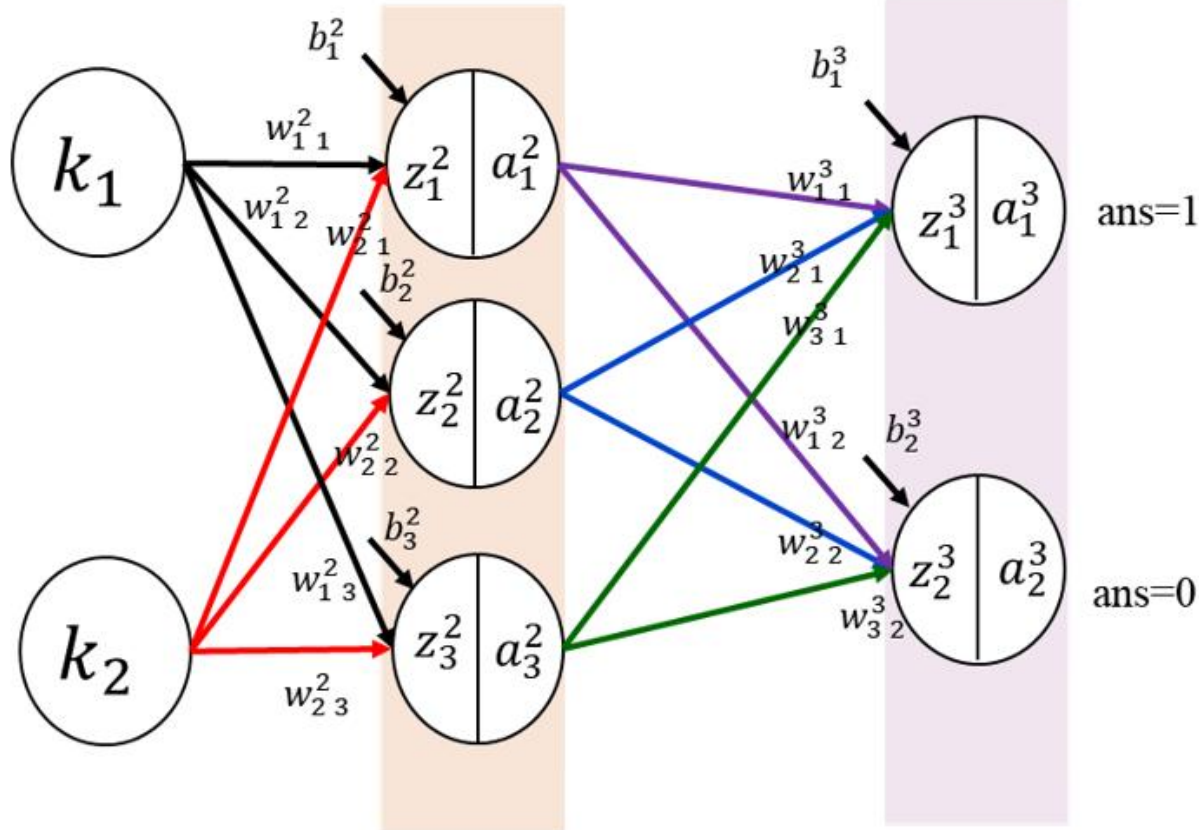
Celem jest ustalenie, czy mężczyźni spodoba się ten owoc, czy nie.

- Istnieją cztery rodzaje owoców: **pomarańcza**, **cytryna**, **ananas** i **japońska persymona**.
- Mężczyzna spożywa te cztery rodzaje owoców i **ocenia poziom słodczy i kwaskowości** owoców w skali od 0 do 9.
- Po ustaleniu poziomu słodczy k_1 i poziomu kwaśności k_2 decyduje on, które owoce lubi, a których nie lubi.
- Założmy więc, że owoce, które lubi, to $[t_1, t_2] = [1, 0]$, a owoce, których nie lubi, to $[t_1, t_2] = [0, 1]$

Owoc n	Słodkość $k_1[n]$	Kwaśność $k_2[n]$	Ocena ogólna	Supervisor $[t_1, t_2]$
Pomarańcza	8	8	Lubi	$[1, 0]$
Cytryna	8	5	Nielubi	$[0, 1]$
Ananas	5	8	Nielubi	$[0, 1]$
Persymona	5	5	Nielubi	$[0, 1]$

Indeksy:

i	Dwóch wejść	$i = 1 \dots 2$
j	Trzech neuronów warstwy ukrytej	$j = 1 \dots 3$
m	Dwóch neuronów warstwy wyjściowej	$m = 1 \dots 2$

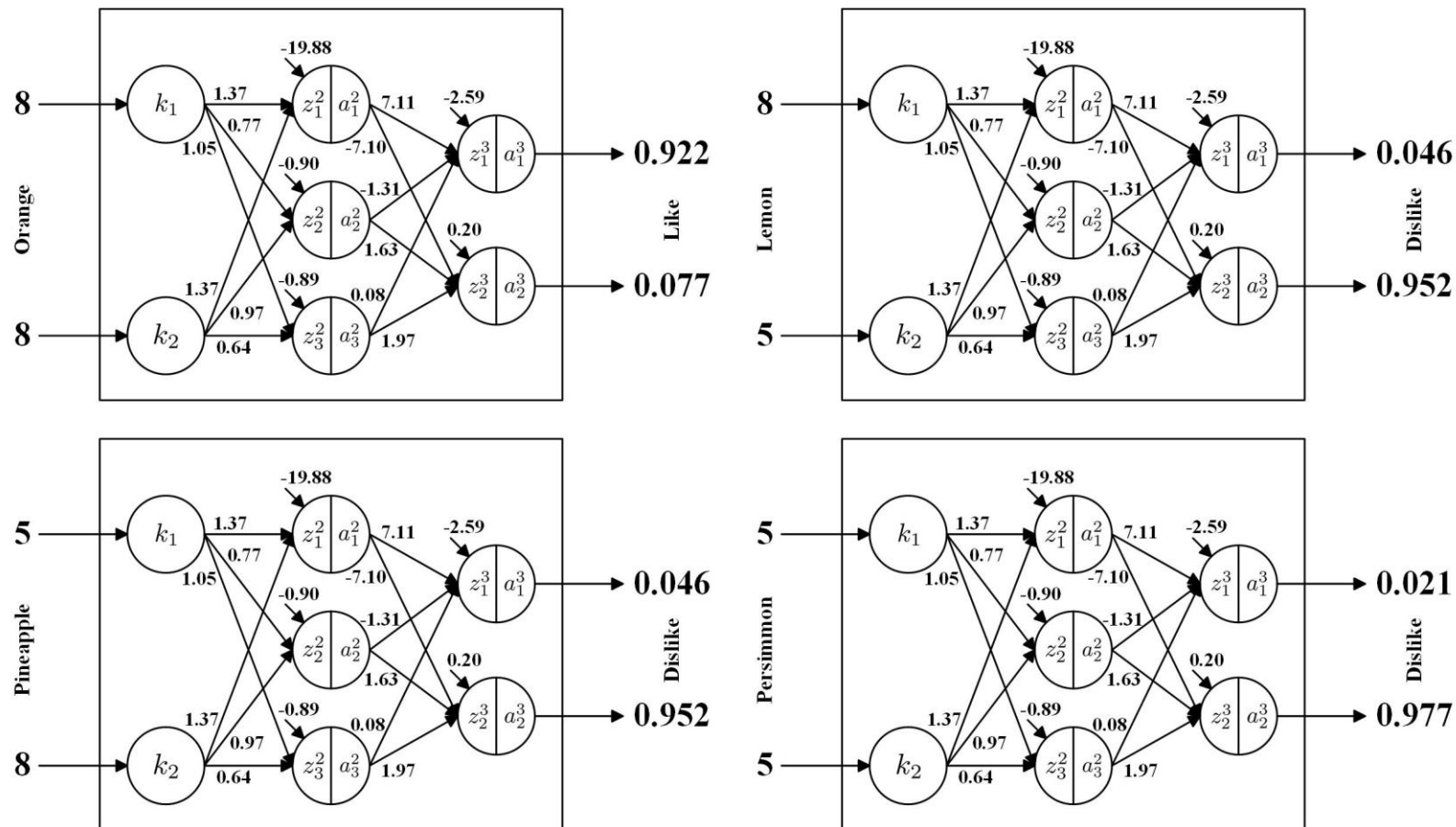


Powyższy rysunek przedstawia model sieci neuronowej wraz z poszczególnymi parametrami w poszczególnych warstwach. Definicje poszczególnych parametrów przedstawiono poniżej:

- k_i – sygnały wejściowe
- w_{ij}^2 – wagi z wejść do warstwy ukrytej
- b_j^2 – odchylenie dla warstwy ukrytej
- z_j^2 – sygnały wejściowe do warstwy ukrytej
- a_j^2 – wyjścia z warstwy ukrytej
- w_{jm}^3 – wagi z warstwy ukrytej do wyjściowej
- b_m^3 – odchylenie dla warstwy wyjściowej
- z_m^3 – sygnały wejściowe do warstwy wyjściowej
- a_m^3 – wyjścia z warstwy wyjściowej

Z opisu tego procesu wynika, że obliczenia dla poszczególnych neuronów są od siebie niezależne. Dzięki temu w sprzęcie komputerowym możemy obliczać wartości wszystkich neuronów równolegle, w tym samym czasie.

Próbki wejściowe są również od siebie niezależne. Dzięki temu możemy obliczać każdą próbkę równolegle, w tym samym czasie. To tak, jakbyśmy mieli cztery identyczne sieci, które wykonują obliczenia na różnych próbkach wejściowych. W ten sposób wszystkie wyniki są generowane jednocześnie.



Tę równoległość obliczeń w neuronach i sieci można przedstawić w postaci macierzy. Obliczenia te można wykonać poprzez mnożenie macierzy. Oto poszczególne etapy:

1. Uzupełnianie danych wejściowych: $K_p = \begin{bmatrix} 8 & 8 & 5 & 5 \\ 5 & 8 & 5 & 8 \\ 1 & 1 & 1 & 1 \end{bmatrix}$

2. Warstwa 2, mnożenie macierzy:

$$\mathbf{Z}_2 = \mathbf{WB}_2 * \mathbf{K}_p$$
$$\mathbf{Z}_2 = \begin{bmatrix} 1.37 & 1.37 & -19.88 \\ 0.77 & 0.97 & -0.9 \\ 1.05 & 0.64 & -0.89 \end{bmatrix} * \begin{bmatrix} 8 & 8 & 5 & 5 \\ 5 & 8 & 5 & 8 \\ 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 2.04 & -2.07 & -2.07 & -6.18 \\ 13.02 & 10.11 & 10.71 & 7.8 \\ 12.63 & 10.71 & 9.48 & 7.56 \end{bmatrix}$$

3. Warstwa aktywacyjna 2:

$$\sigma(\mathbf{Z}_2) = \mathbf{A}_2$$
$$\sigma\left(\begin{bmatrix} 2.04 & -2.07 & -2.07 & -6.18 \\ 13.02 & 10.11 & 10.71 & 7.8 \\ 12.63 & 10.71 & 9.48 & 7.56 \end{bmatrix}\right) = \begin{bmatrix} 0.884 & 0.112 & 0.112 & 0.002 \\ 1 & 0.999 & 0.999 & 0.999 \\ 1 & 0.999 & 0.999 & 0.999 \end{bmatrix}$$

4. Wykonanie dalszego wypełniania warstwy wyjściowej 2 oraz warstwy mnożenia macierzy 3 i warstwy aktywacyjnej 3:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}$$

Napisz prosty kod w języku Python, aby wykonać te obliczenia.

Reprezentacja stałoprzecinkowa

Zastosowana reprezentacja w systemie liczb stałoprzecinkowych:

- 1 bit - znak
- 5 bitów – część całkowita
- 10 bitów – część ułamkowa

Przykład:

- $16'b0\ 00000\ 1000000000 = 0.5$

Użyj funkcji w języku `python` do konwersji na liczbę stałoprzecinkową i odwrotnie.

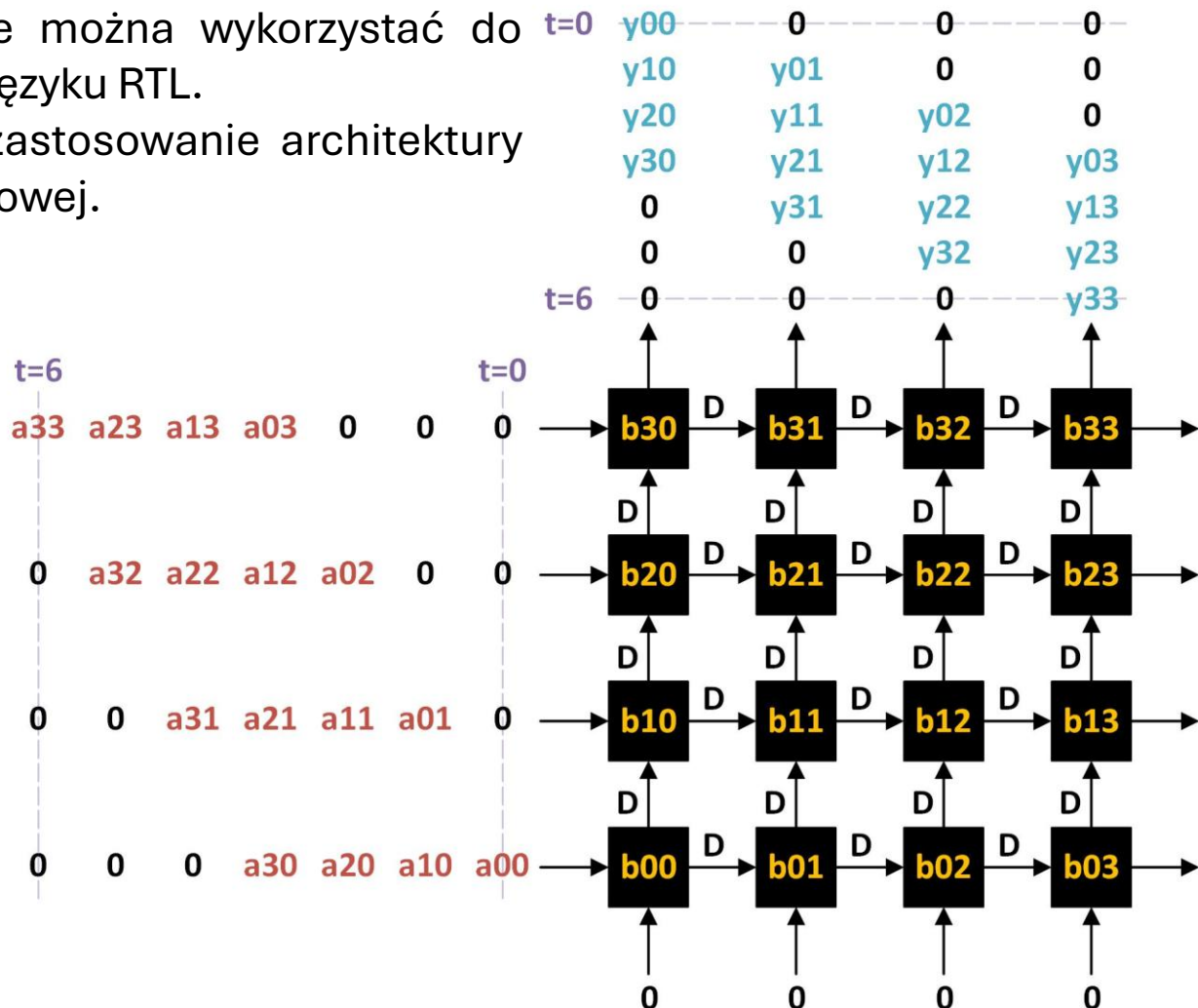
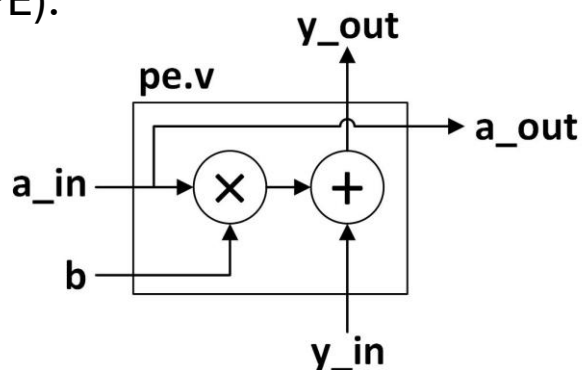
Macierz skurczowa

Istnieje wiele architektur, które można wykorzystać do realizacji mnożenia macierzy w języku RTL.

W tym przypadku rozważamy zastosowanie architektury macierzy systolicznej lub skurczowej.

Macierz skurczowa (*systolic*) to architektura przepływu danych zbudowana z wielu identycznych komórek.

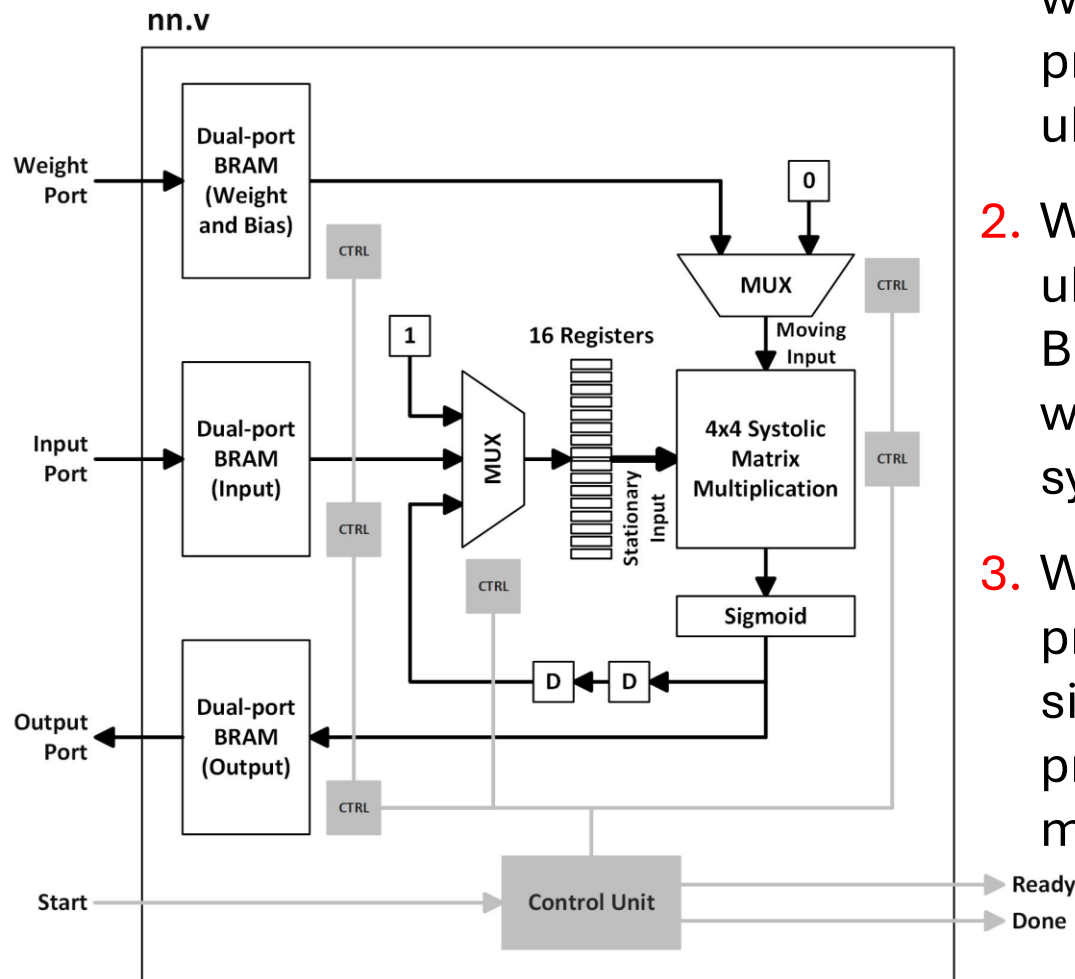
Komórka ta nazywana jest elementem przetwarzającym (PE).



Moduł szczytowy

Oto przebieg przepływu danych:

1. Dane wejściowe są odczytywane z wejścia pamięci BRAM, a następnie przesyłane do wejścia stacjonarnego układu systolicznego.
2. Wagi i przesunięcia dla warstwy ukrytej są odczytywane z pamięci BRAM, a następnie przesyłane do wejścia ruchomego układu systolicznego.
3. Wynik z modułu systolicznego jest przetwarzany przez moduł sigmoidalny, a następnie wynik jest przesyłany do wejścia stacjonarnego modułu systolicznego.

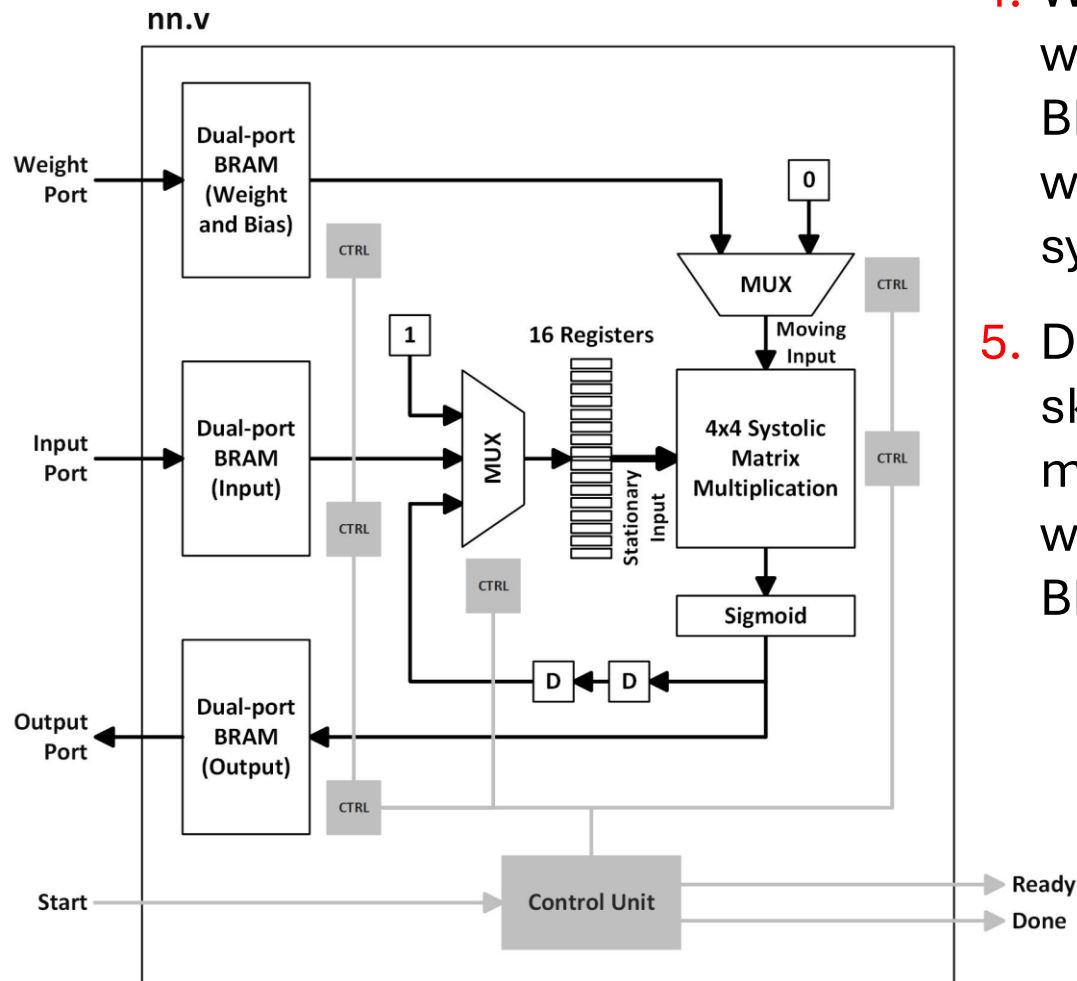


Moduł szczytowy

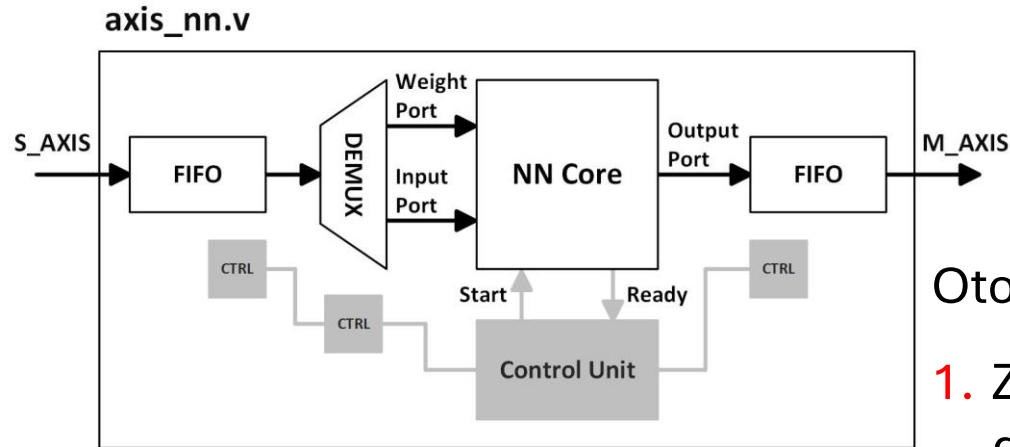
Ciąg dalszy przebieg przepływu danych:

4. Wagi i przesunięcia dla warstwy wyjściowej są odczytywane z pamięci BRAM, a następnie przesyłane do wejścia ruchomego układu systolicznego.

5. Dane wyjściowe z modułu skurczowego są przetwarzane przez moduł sigmoidalny, a następnie wynik jest przesyłany do wyjścia BRAM.



Moduł AXI-Stream



Oto przebieg przepływu danych:

1. Zarówno dane dotyczące wagi, jak i dane wejściowe są przesyłane strumieniowo przez port `S_AXIS`.
2. Demultiplekser rozdziela dane, kierując część do portu wagi, a część do portu wejściowego modułu sieci neuronowej.
3. Jednostka sterująca uruchamia moduł sieci neuronowej i czeka na zakończenie jego pracy.
4. Dane wyjściowe są przesyłane strumieniowo do portu `M_AXIS`.

Projekt RTL Vivado

Projekt składa się z sześciu plików źródłowych:

- `axis_nn.v`
- `nn.v`
- `pe.v`
- `register.v`
- `sigmoid.v`
- `systolic.v`

i dwóch plików danych:

- `indata.mem`
- `weights.mem`

Słowniczek:

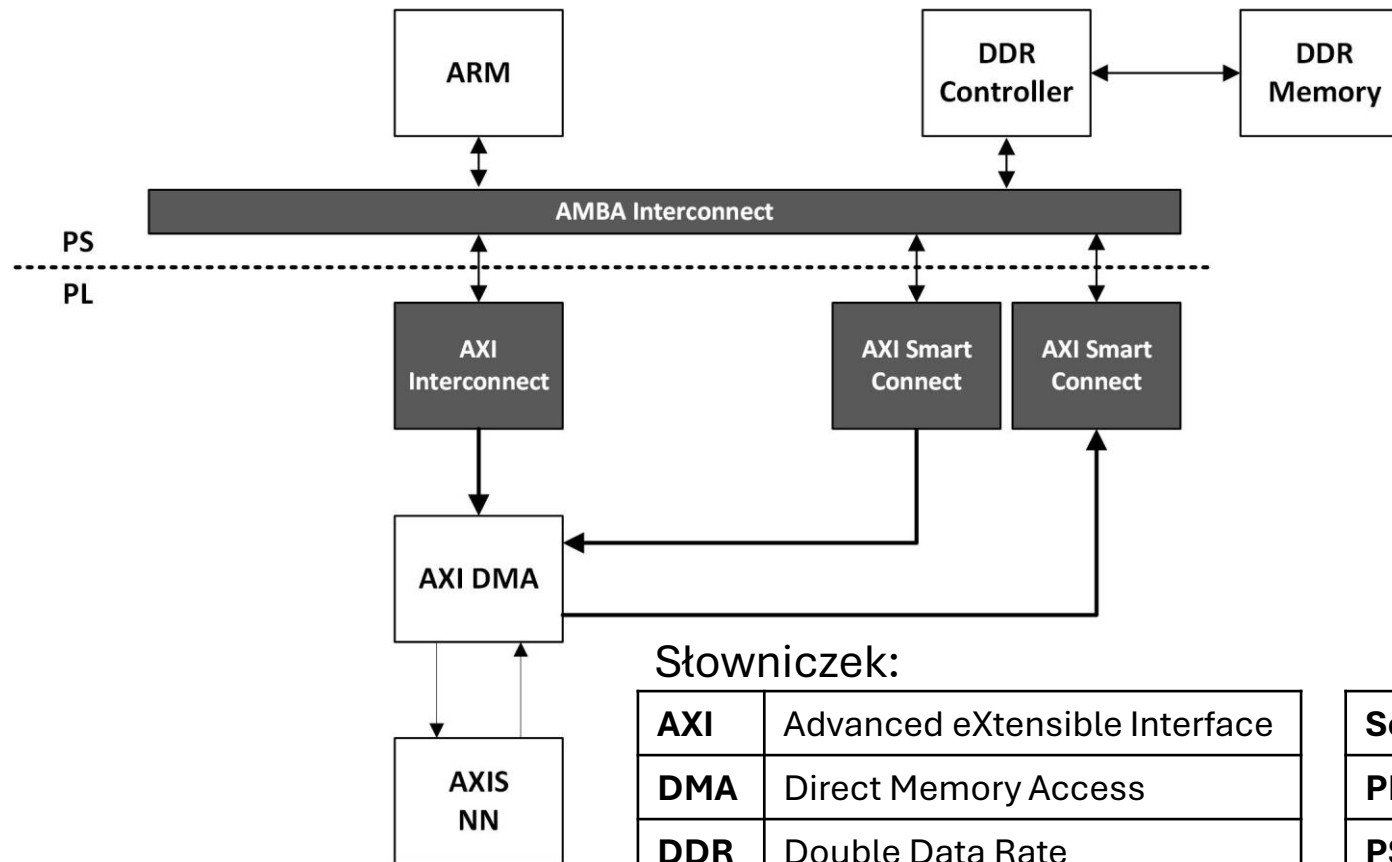
RTL	Register Transfer Level
BD	Block Design

```
axis_nn (axis_nn.v) (3)
├── reg_s2mm_valid : register (register.v)
├── reg_s2mm_last : register (register.v)
└── nn_0 : nn (nn.v) (32)
    ├── reg_b00 : register (register.v)
    ├── reg_b01 : register (register.v)
    ├── reg_b02 : register (register.v)
    ├── reg_b03 : register (register.v)
    ├── reg_b10 : register (register.v)
    ├── reg_b11 : register (register.v)
    ├── reg_b12 : register (register.v)
    ├── reg_b13 : register (register.v)
    ├── reg_b20 : register (register.v)
    ├── reg_b21 : register (register.v)
    ├── reg_b22 : register (register.v)
    ├── reg_b23 : register (register.v)
    ├── reg_b30 : register (register.v)
    ├── reg_b31 : register (register.v)
    ├── reg_b32 : register (register.v)
    ├── reg_b33 : register (register.v)
    ├── dut : systolic (systolic.v) (73)
    ├── sigmoid_0 : sigmoid (sigmoid.v) (1)
    ├── sigmoid_1 : sigmoid (sigmoid.v) (1)
    ├── sigmoid_2 : sigmoid (sigmoid.v) (1)
    ├── sigmoid_3 : sigmoid (sigmoid.v) (1)
    ├── reg_sig_00 : register (register.v)
    ├── reg_sig_01 : register (register.v)
    ├── reg_sig_10 : register (register.v)
    ├── reg_sig_11 : register (register.v)
    ├── reg_sig_20 : register (register.v)
    ├── reg_sig_21 : register (register.v)
    ├── reg_sig_30 : register (register.v)
    ├── reg_sig_31 : register (register.v)
    ├── reg_sig_valid_0 : register (register.v)
    ├── reg_sig_valid_1 : register (register.v)
    └── reg_sig_valid_2 : register (register.v)
```

Projekt blokowy BD Vivado

Poniższy rysunek przedstawia ogólną strukturę układu SoC akceleratora sieci neuronowej.

Wykorzystujemy moduł IP AXI DMA, który przekształca dane mapowane w pamięci na dane strumieniowe i odwrotnie.



Słowniczek:

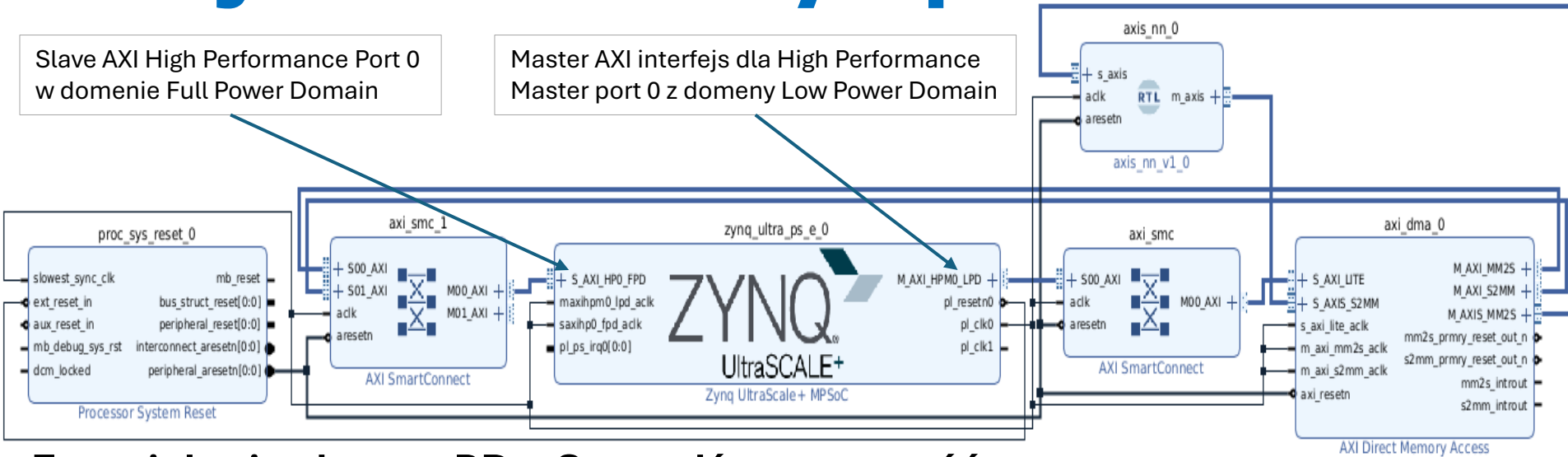
AXI	Advanced eXtensible Interface
DMA	Direct Memory Access
DDR	Double Data Rate
ARM	Advanced RISC Machines

SoC	System on Chip
PL	Programmable Logic
PS	Programming System
IP	Intellectual Properties

Projekt BD dla Zynq-UltraScale+

Slave AXI High Performance Port 0
w domenie Full Power Domain

Master AXI interfejs dla High Performance
Master port 0 z domeny Low Power Domain



Zaprojektuj schemat BD → Sprawdź poprawność

Wygeneruj produkt (poza kontekstem dla każdego rdzenia IP)

Zapakuj wynik do modułu HDL i ten moduł ustaw jako szczytowy

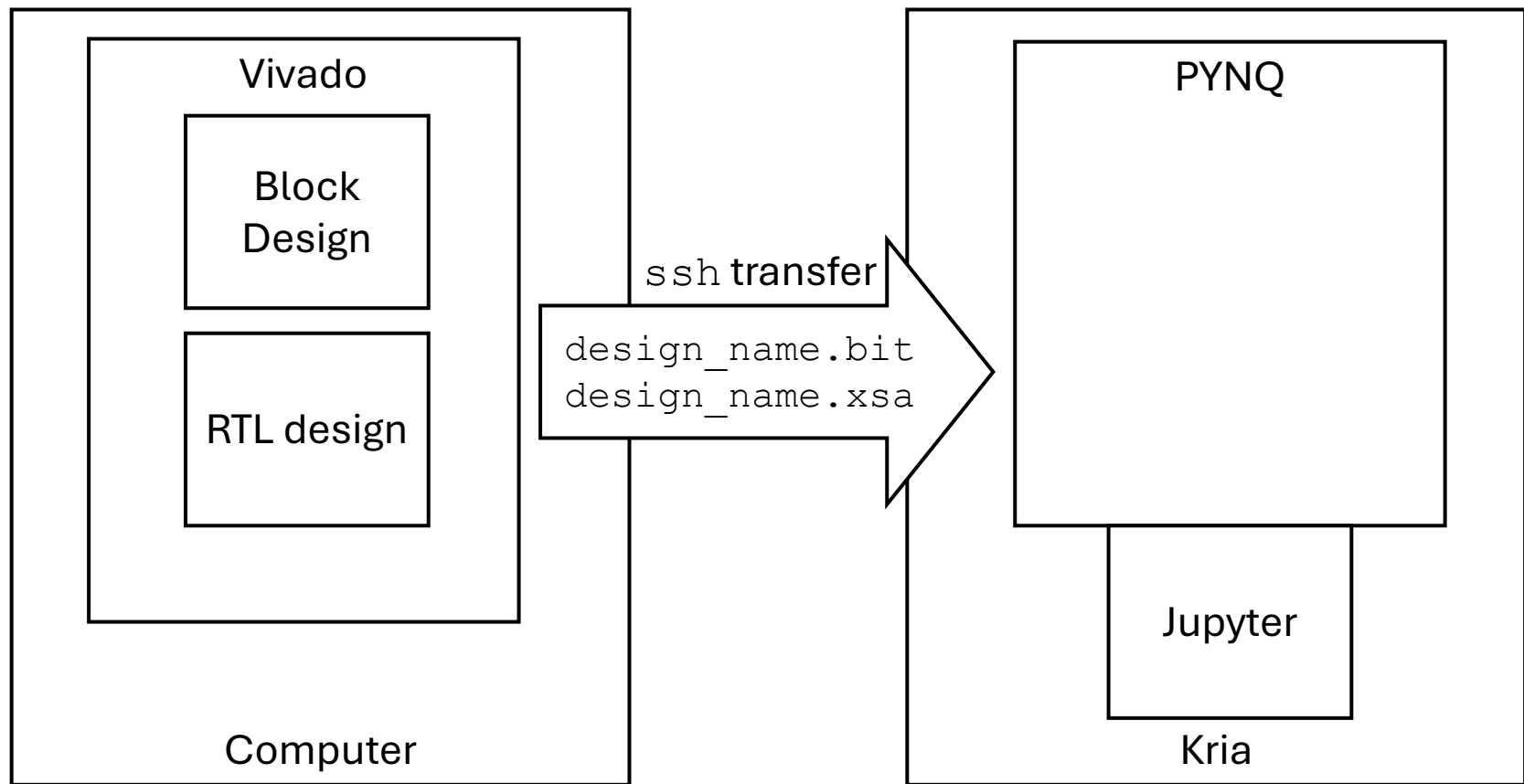
Zsyntetyzuj → Zaimplementuj → Wygeneruj strumień bitów

Wyeksportuj plik .xsa (File → Export → Export Hardware (include bitstream))

Skopiuj niezbędne pliki do modułu Kria z systemem Ubuntu-PYNQ

Słowniczek niektórych oznaczeń:

M_AXI_MM2S	Master AXI interface do odczytu danych z pamięci (Memory-Mapped) i przesyłania ich jako strumień (Stream)
M_AXI_S2MM	Master AXI interface do zapisu danych ze strumienia (AXI4-Stream) do pamięci (Memory-Mapped)
M_AXIS_MM2S	Master AXI-Stream interface, który wysyła dane pobrane z pamięci (MM → Stream)



<code>ann_ps.bit</code>	Xilinx BIT data – from <code>ann_ps_wrapper</code> ; UniqueID=97f3f0cb; ParentID=00000000; UserID=0xFFFFFFFF; Version=2025.1; SW_CRC=eef88db0 - for xck26-sfvc784-2LV-c - built 2025/12/08(23:21:13) - data length 0x76fbbc
<code>ann_ps.tcl</code>	ASCII text
<code>ann_ps.hwh</code>	XML 1.0 document , Unicode text, UTF-8 (with BOM) text, with very long lines (1180)

Pliki ważne dla PYNQ

Plik **Hardware Handoff (.hwh)** zawiera kluczowe szczegóły projektu, takie jak adresy pamięci i konfiguracje układów IP, niezbędne do połączenia oprogramowania, np. PYNQ lub Vitis, ze sprzętem FPGA.

Plik ten jest generowany przez narzędzie Vivado. Wygenerowanie pliku .hwh powoduje spakowanie projektu IP Integrator, co pozwala oprogramowaniu zrozumieć układ sprzętowy i komunikować się z urządzeniami peryferyjnymi.

Trzeba skopiować ten plik z katalogu:

```
desname.gen/sources_1/bd/bdname/hw_handoff
```

do Jupytera na Krii, a następnie zmienić mu nazwę tak aby była taka sama jak dwóch pozostałych plików.

Pliki ważne dla PYNQ

Plik strumienia bitów .bit zawiera informacje z listy sieciowej dotyczące sposobu programowania bitów CCL (pamięć SRAM) w układach CMOS Configuration Latches w układzie FPGA:

- w tablicach LUT (mapowanie, grupowanie i rozmieszczenie),
- w blokach CLB (routing lokalny),
- w blokach SB i CB (routing globalny).

Trzeba skopiować ten plik z katalogu:

```
desname.runs/impl_1/
```

do Jupytera na Krii.

Pliki ważne dla PYNQ

TCL Tool Comman Language skrypt do odtworzenia całego projektu `.tcl`

Trzeba skopiować ten plik z katalogu:

`desname.runs/impl_1/`

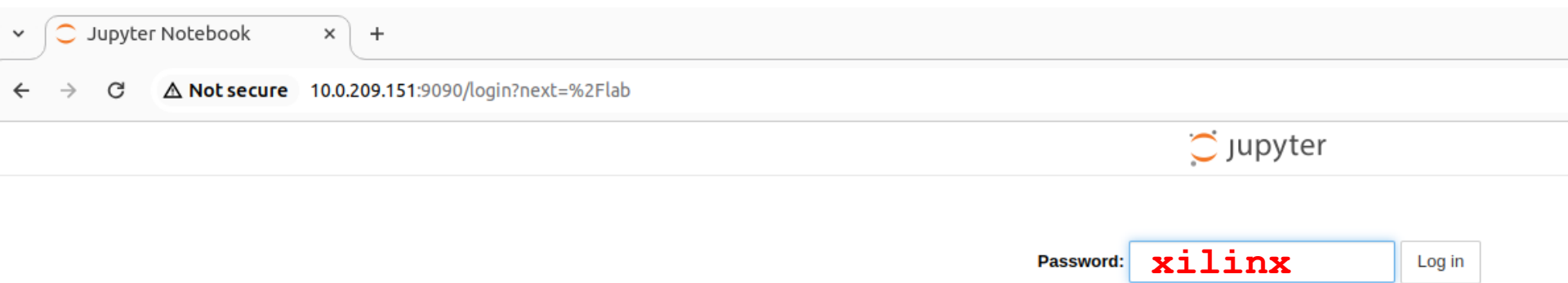
do Jupytera na Krii.

```
$ vivado -script desing_name.tcl &
```

To jest polecenie przywracające projekt o nazwie `design_name`

W naszym przypadku nie musimy tego robić.

Notatnik Jupyter dla PYNQ



```
scp desname.runs/impl_1/*.tcl  
ubuntu@<kria_IP_number>:.
```

```
scp  
desname.gen/sources_1/bd/bdname/hw_handoff  
/bdname.hwh ubuntu@<kria_IP_number>:.
```

```
scp desname.runs/impl_1/bdname  
_wrapper.bit ubuntu@<kria_IP_number>:.
```

desname	Vivado project name
bdname	Block Design name

Można też zrobić copy-and-paste w notatniku Jupyter pod systemem operacyjnym

Plik ważny dla Vitis

ale zbędny dla PYNQ

Xilinx Shell Archive (.xsa) to nowsza wersja pliku .hwh, będąca kontenerem zawierającym następujące informacje:

- Jeden lub więcej plików .hwh
- Informacje o wersji narzędzia AMD Vivado™, części i tagu płytki
- IP – instancja, nazwa, VLNV i parametry
- Informacje o mapie pamięci procesorów
- Informacje o łączności wewnętrznej (w tym przerwania, zegary itp.) oraz informacje o portach zewnętrznych
- Pliki BMM/MMI i BIT
- Pliki sterowników użytkownika i HLS
- Pliki drzewa urządzeń
- Inne pliki metadanych